SolariNet Certificate Operations

Candidate operations manual · refreshed 24 October 2026 · F-01 CRL lifecycle · F-10 trust and issuance

> **Candidate only — do not enable a rollout from this manual yet.** The generator, publisher, peer updater, acknowledgement path, and dashboard view exist in the local candidate. The HTTPS endpoint, host configuration, peer activation, rollout policy, and deployment are not installed or verified. This document describes the reviewed procedure for a later staged rollout; it is not evidence of a live service.

1. What a revocation means

The MariaDB revocation ledger is authoritative. A ledger entry alone does not change a TLS handshake. Effective revocation requires a fresh CA-signed CRL, authenticated HTTPS distribution, validation and activation on every relevant peer, and a subsequent authenticated report receipt. The current candidate receipt proves only that the reporting client or monitor agent included the local updater's activation marker. It does not attest every service on that host.

Keep the immutable root bundle separate from the generated active bundle. Never copy the CA private key to a publisher, proxy, dashboard, or peer. Do not expose a CRL endpoint over plaintext HTTP.

The dashboard counts only client and monitor reporting agents. It does not prove all host services loaded the CRL.

2. Components and access boundaries

Component	Runs as / reads	Writes or authority
CRL generator	`solari`; CA key, CA certificate, revocation ledger, issuance-inventory gate	Signs a bounded-lifetime CRL and advances per-issuer generation state
Static publisher	Dedicated unprivileged account; generated CRL and public CA certificate	Publishes only a verified issuer-fingerprint-named CRL
Peer updater	Root oneshot; trusted HTTPS endpoint, immutable roots, issuer certificate	Replaces one active bundle and restarts one allowlisted SolariNet service
Reporter/ server	Existing authenticated durable report; enrolled certificate identity	Stores latest exact-generation receipt; receipt DB failure does not reject the persisted report
Dashboard	Read-only API; generated generation, node state and receipt rows	Displays latest generation and client/monitor receipt status

3. Preconditions before any installation

1. Apply and verify the candidate migration set, including migration 035, using the approved migration process. Reconcile and attest the certificate issuance inventory before generating a CRL.
2. Confirm the issuer certificate and private key match. Keep the key owned by `solari` with mode `0600`. Do not grant the publisher, peer updater, web server, or dashboard access to it.
3. Establish a peer-trusted HTTPS hostname and certificate. Verify an unauthenticated peer GET serves exactly the generated CRL, and that the same path is unavailable over HTTP.
4. Review the exact service allowlist, active bundle paths, systemd sandbox, database grants, rollback behavior, and recovery access for each target host.
5. Use a staged, non-production peer first. Keep rollout policy fail-closed until the intended missing-CRL behavior is separately tested and approved.

4. Database accounts

Dashboard read account

The dashboard's read-only DB account also needs these CRL table reads in addition to its existing `node` privileges:

```
GRANT SELECT ON solarinet.certificateCrlState TO 'SOLARINET_DASHBOARD'@'DASHBOARD_HOST';
GRANT SELECT ON solarinet.certificateCrlAcknowledgement TO 'SOLARINET_DASHBOARD'@'DASHBOARD_HOST';
```

Server receipt account

Add only these rights to the existing SolariNet server account after migration 035 is present. Replace both account placeholders on the database host; do not paste secrets into commands or repository files.

```
GRANT SELECT ON solarinet.certificateCrlState TO 'SOLARINET_SERVER'@'SERVER_HOST';
GRANT SELECT, INSERT, UPDATE ON solarinet.certificateCrlAcknowledgement TO
'SOLARINET_SERVER'@'SERVER_HOST';
```

Acknowledgement persistence is best-effort. Missing migration or rights are logged and later reports retry the local receipt. The dashboard remains pending or stale until a valid matching row is recorded.

Generator account

```
GRANT SELECT ON solarinet.certificateInventoryState TO 'solari_crl'@'CRL_HOST';
GRANT SELECT ON solarinet.certificateRevocation TO 'solari_crl'@'CRL_HOST';
GRANT SELECT, INSERT, UPDATE ON solarinet.certificateCrlState TO 'solari_crl'@'CRL_HOST';
```

Use the narrowest DB host identity available and a generated password. Store client credentials in `/etc/solarinet/crl-generator.my.cnf`, readable only by `solari`:

```
[client]
host=DB_HOST
port=3306
user=solari_crl
password=REPLACE_ON_HOST_ONLY
```

```
sudo install -d -o root -g solari -m 0750 /etc/solarinet
sudoedit /etc/solarinet/crl-generator.my.cnf
sudo chown root:solari /etc/solarinet/crl-generator.my.cnf
sudo chmod 0640 /etc/solarinet/crl-generator.my.cnf
```

5. Install the CA-side generator and publisher

Install only reviewed release files. SolariNet C server daemon connections initialize their MariaDB session timezone to UTC on connect and reconnect; the generator uses UTC for ledger reads and generation writes. Xenon's active monitoring MariaDB was observed in `SYSTEM` mode with the host on UTC, and its `NOW()` matched `UTC_TIMESTAMP()`; no historical timestamp offset was observed there. Verify the actual configured database host before deployment. The generator service runs every six hours; a separate one-minute publisher timer exposes the generated CRL. Pre-publication failures preserve the previous CRL. There is a short non-transactional boundary after the atomic CRL replacement and before the generator advances the DB generation row; if that DB commit fails, the file can be one CRL number ahead of acknowledgement state.

```
sudo install -d -o root -g root -m 0755 /usr/local/libexec/solarinet
sudo install -o root -g root -m 0755 deploy/crl/generate_crl.py /usr/local/libexec/solarinet/
generate_crl.py
sudo install -o root -g root -m 0644 deploy/crl/solari-crl-generate.service /etc/systemd/system/
sudo install -o root -g root -m 0644 deploy/crl/solari-crl-generate.timer /etc/systemd/system/
sudo install -o root -g root -m 0644 deploy/crl/solari-crl-publisher.sysusers.conf /etc/sysusers.d/
solari-crl-publisher.conf
sudo systemd-sysusers /etc/sysusers.d/solari-crl-publisher.conf
sudo install -o root -g root -m 0755 deploy/crl/publish_crl.py /usr/local/libexec/solarinet/
publish_crl.py
sudo install -o root -g root -m 0644 deploy/crl/solari-crl-publish.service /etc/systemd/system/
sudo install -o root -g root -m 0644 deploy/crl/solari-crl-publish.timer /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemd-analyze verify /etc/systemd/system/solari-crl-generate.service /etc/systemd/system/
solari-crl-generate.timer /etc/systemd/system/solari-crl-publish.service /etc/systemd/system/solari-
crl-publish.timer
```

Before enabling timers, check schema and inventory readiness, file ownership, account grants, and key/certificate match. Run one generation and inspect the issuer fingerprint, number, revoked count, CRL fingerprint, and expiry:

```
sudo systemctl start solari-crl-generate.service
sudo systemctl status solari-crl-generate.service --no-pager
sudo openssl crl -in /var/lib/solari-crl/current.pem -noout -verify \
  -CAfile /var/lib/solarinet/pki/ca.pem -crlnumber -lastupdate -nextupdate
sudo systemctl enable --now solari-crl-generate.timer solari-crl-publish.timer
```

Configure a static-only path inside a peer-trusted HTTPS virtual host using `deploy/crl/apache-crl-location.conf.example` or an equivalent rule. Do not expose directory listings, dotfiles, signing material, or database credentials.

6. Configure a peer updater

One updater instance handles one issuer and one service. Keep the immutable base bundle free of CRLs and include the issuer certificate. The timer first runs two minutes after boot or at the next hour with up to five minutes of jitter. Seed the active bundle from the configured base bundle before setting the service's `tls.caFile`; otherwise an early service restart can fail because the file does not yet exist. The per-service configuration contains public paths and a URL, not a secret:

```
CRL_URL=https://crl-host.example/solarinet/crl/ISSUER_SHA256.crl.pem
BASE_CA_BUNDLE=/etc/solarinet/trust-roots.pem
ISSUER_CA=/etc/solarinet/issuing-ca.pem
SOLARI_SERVICE=solari-client.service
```

Replace the issuer placeholder with the lowercase SHA-256 fingerprint printed by the generator. The endpoint hostname must match its HTTPS certificate. Seed the active bundle using the same base bundle path, then configure `tls.caFile` to the active path. Install the updater only after the endpoint has been independently verified from the peer.

The updater refuses every HTTP redirect. It uses the operating system CA store by default. For direct invocations, `--https-ca-file` selects an exclusive trust file; it does not add to the system roots.

```
sudo install -d -o root -g root -m 0755 /var/lib/solari-crl-active
sudo install -o root -g root -m 0644 /etc/solarinet/trust-roots.pem /var/lib/solari-crl-active/
client.pem
sudo install -d -o root -g root -m 0755 /usr/local/libexec/solarinet
sudo install -o root -g root -m 0755 deploy/crl/activate_crl.py /usr/local/libexec/solarinet/
activate_crl.py
sudo install -o root -g root -m 0644 deploy/crl/solari-crl-activate@.service /etc/systemd/system/
sudo install -o root -g root -m 0644 deploy/crl/solari-crl-activate@.timer /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemd-analyze verify /etc/systemd/system/solari-crl-activate@.service /etc/systemd/system/
solari-crl-activate@.timer
sudo systemctl enable --now solari-crl-activate@client.timer
```

The updater verifies HTTPS, issuer, signature, expiry, and monotonic number; rejects rollback or conflicting same-number CRLs; limits files to 4 MiB and validation to 120 seconds; serializes work per active bundle; and restarts only an allowlisted service. If restart/active verification fails, it restores the previous bundle and attempts recovery. A failed rollback requires manual intervention. The activation service has a 360-second outer timeout to leave 240 seconds for bounded recovery.

7. Verify reports and dashboard status

After a successful restart, the updater atomically writes `<active-bundle>.activated` as `issuer-sha256:number:crl-sha256`, mode `0644`. The client/monitor reporter includes this public marker in its authenticated durable report. The server records it only for the latest generated, unexpired generation. Stale, malformed, or unwriteable optional receipts do not reject the ordinary report.

The Certificates screen shows the latest generated issuer and CRL number, expiry, and peer receipt states. **Current** means an exact issuer/number/fingerprint receipt for an unexpired generation and a check-in within 15 minutes. **Expired generation** means the receipt matches the latest CRL, but that CRL has passed its next-update time or the expiry value is missing. **Stale** means a prior receipt does not match the latest generation. **Pending** means a recently seen peer has no receipt. **Offline** means no check-in within 15 minutes or node state is down. The denominator includes client and monitor nodes whose current leaf issuer matches that generation. Excluded agents are listed separately: an unknown issuer means no current leaf issuer is recorded; a different issuer means it is recorded but does not match. Server-role peers and other services on each host remain outside this status view.

> **Interpret carefully:** a current marker is evidence from one reporting agent, not cryptographic proof of runtime process state or host-wide convergence. A generated CRL is not necessarily already served, and a served CRL is not proof it was activated.

8. Recovery and rollback

Observed result	Operator action
Generator fails before output replacement	Read the systemd journal. Check ledger completeness, migration state, CA key/certificate match, DB rights, and system clock. Do not replace the last good CRL manually.
CRL file is one number ahead of DB state	A DB commit may have failed just after atomic file replacement. Restore DB availability, rerun the generator to allocate a higher number, and verify the output file and `certificateCrlState` agree before reading dashboard status.
Publisher fails or endpoint returns an old object	Verify generator output and publisher service/timer. Confirm the HTTPS static path serves the expected fingerprint-named file. Do not enable peer timers until this is corrected.
Updater rejects signature, issuer, number, or expiry	Keep the current trust bundle. Inspect endpoint URL, issuer fingerprint, current CRL metadata, and system clock; correct the source instead of bypassing validation.
Service restart fails	The updater restores the prior bundle and retries the prior service state. Inspect unit logs and verify the active bundle. If rollback reports failure, stop automation and restore the known-good bundle manually before restarting.
Dashboard shows pending/ stale	Check peer check-in, receipt file ownership/mode, reporter config, server migration 035 and grants, and whether the peer has fetched the latest generated generation.
Dashboard says acknowledgement data unavailable	Check API logs, migration 035, and read access. The API fails soft; this state is not evidence that all peers are current.

Do not delete acknowledgement rows to make a dashboard count appear healthy. To roll back an updater, disable its timer, restore the previous reviewed active bundle and matching service configuration, then restart and verify the exact target service. Preserve logs and the activation marker for diagnosis.

9. Browser trust and web-server certificates

The browser trust page publishes only the public CA root and setup instructions. It does not issue certificates or contain a signing key. Verify the root SHA-256 fingerprint through a trusted channel independent of the portal before installation.

Client	Functional setup	Verification
Safari on iPhone/ iPad; Firefox on iOS	Transfer the unsigned root profile over AirDrop or Files after verifying its fingerprint. Install it in Settings, then enable full trust under Settings → General → About → Certificate Trust Settings. Device management is not required. An Apple Developer key does not establish TLS trust and is not used.	Open a known internal HTTPS service, confirm its hostname matches the certificate SAN, and confirm the browser shows no certificate warning.
Safari and Zen on macOS	Install the root in the System keychain after verifying its fingerprint. Scope trust to SSL and basic X.509 validation with `sudo security add-trusted-cert -d -r trustRoot -p ssl -p basic -k /Library/Keychains/System.keychain root_ca.pem`. Firefox-family browsers may need their system-root option enabled.	Restart each browser and test an internal HTTPS hostname. Remove the root by its fingerprint in Keychain Access if trust must be withdrawn.
Firefox on Linux	Install the root and Firefox enterprise policy from the trust package. Confirm the policy in `about:policies`; Flatpak/Snap installations can use a different policy path.	Restart Firefox and test the internal HTTPS hostname and its SAN.
Firefox on Android	Install the root through Android's certificate settings after verifying its fingerprint. Firefox 120 or later can use third-party roots from the OS store; check the actual browser build.	Restart Firefox and open a known internal HTTPS service without bypassing a warning.

On first use, the portal itself cannot establish trust in its own HTTPS connection. Transfer and fingerprint-check the public root first; then visit the portal over HTTPS.

Web-server ACME lifecycle

The candidate ACME helper uses Certbot and HTTP-01 with an isolated private-CA store at `/etc/letsencrypt-solarinet`. It requires Python 3.10 or later, apt-packaged Certbot 4.0.0 or later, and OpenSSL 3.0 or later for short-lived renewal behavior and isolated chain verification. Certbot still reads `/etc/letsencrypt/cli.ini` and its user's XDG `letsencrypt/cli.ini` with a dedicated store; command-line values take precedence. The helper clears global pre/post hooks, disables hook-directory executables, and supplies its own deploy hook. Review both default files for other settings before use; a legacy `renew-hook` conflicts with the helper deploy hook. The helper validates the exact registered DNS names, server-auth usage, a minimum six-day lifetime, chain, and key match; writes a versioned certificate/key pair; switches `TARGET/current` atomically; runs the service configuration test; and reloads the server. A failed check or reload restores the prior pointer and reports up to five diagnostic lines. Renewal verifies after Certbot exits that every registered live lineage matches the active release, so a deploy-hook error is visible to systemd. The key is private to the host and is not copied to the CA, dashboard, or Git.

> **Do not issue against the live CA yet.** Its ACME provisioner has no explicit challenge list or name policy. Self-hosted step-ca applies X.509 name policy authority-wide, which also affects JWK and S/MIME issuance; SolariNet node enrollment uses a separate mTLS CA. Audit step-ca identities before adding policy. Use only the disposable ACME test CA until this audit and the name-restricted pilot pass.

1. Inventory each HTTPS endpoint, exact SANs, server owner, HTTP-01 route, certificate paths, key-reader group, config-check command, and reload command. Give each service a distinct lineage.
2. Copy `deploy/pki/acme_services.example.json` to a root-owned mode-`0600` registry. It uses reserved `example.invalid` and `ca.example.invalid` placeholders; replace both with the approved disposable CA and test-service values. The registry's commands run as root.
3. Route `/.well-known/acme-challenge/` to `/var/lib/solarinet/acme-webroot/.well-known/acme-challenge/` on the intended HTTP virtual host. Confirm the CA host can retrieve a test challenge over port 80.
4. Run `acme_certificates.py --registry /etc/solarinet/pki/acme_services.json --plan-service SERVICE_ID` to inspect the inert enrollment command. Then run `--issue-service SERVICE_ID` only against the approved disposable CA.
5. After the first certificate exists, update the intended server block to use `TARGET/current/fullchain.pem` and `TARGET/current/privkey.pem`. Test the complete server config and HTTPS handshake.
6. Configure the test ACME provisioner for a seven-day default and no more than a 30-day maximum lifetime. The helper rejects leaves under six days. Certbot 4.0 uses a lifetime-based renewal threshold (half the lifetime for certificates of ten days or less; one third for longer certificates), so twice-daily checks do not issue twice daily. Verify the installed Certbot version and behavior in the pilot. Run `--renew --force-renewal` to rehearse the complete unattended path, including HTTP-01, atomic installation, service validation, and reload. If issuance succeeded but deployment did not, retry the stored lineage with `--deploy-service SERVICE_ID`.
7. Enroll every registered service before adding it to the renewal timer's registry; renewal fails closed if registry lineages and Certbot renewal files differ. Review Certbot's default `cli.ini` files and connect external expiry monitoring. Install and enable the supplied `solarinet-acme-renew.timer` only after forced renewal and rollback rehearsal succeed for every service. It renews certificates only from the dedicated private-CA Certbot store; the host's public Let's Encrypt store stays separate. `UMask=0022` lets the HTTP server read public challenge files; Certbot retains private-key mode `0600`.

Use the documented systemd paths or add matching reviewed write paths. The timer supplies a 20-minute randomized start; Certbot's extra delay is disabled. The service allows 45 minutes for the helper's 30-minute Certbot limit. CA policy must enforce key size, signature algorithm, and the 825-day ceiling; the helper does not. Unexpected release files block deployment. For rollback, stop the timer, atomically point `TARGET/current` to a verified release, then run the service check and reload. The renewal post-check will fail until the Certbot lineage is restored with `--deploy-service` or successfully renewed; keep unattended renewal disabled during recovery.

See deploy/pki/README.md for server snippets, timer settings, and recovery steps. The 30 ACME helper unit tests cover validation and path safety. The isolated integration test exercises step-ca, HTTP-01, Apache, renewal, name-policy rejection, HTTPS, and rollback. Real issuance against Chlorine, browser/device trust, expiry alerts, and rollout remain **unverified**.

10. Current verification and release gates

Checks pass: PKI 65/65, Python compilation, systemd verification, Forgejo workflow parity, and isolated ACME lifecycle integration. Claude Fable approved with no High/Medium findings; optional Low notes are resolved. GitHub run `40057443974` passed all 16 jobs on `5a51e38`, including ACME and scanner/RackWire acceptance. Forgejo's ephemeral runner supervisor and VM are active on Benzene; this candidate's Forgejo run result is still **unverified**. Real Chlorine issuance, browser trust, expiry alerts, peer activation, service cutover, and rollout remain **unverified**.